Prepared for
Leandro Schlottchauer
Andre Brener
Fuul.xyz

Prepared by
Sergio Corrales
Ignacio Sanmillan
Inversive Labs

January 21st 2026

ILAR-2026-02

Fuul Protocol - Solana

Invariant Testing Engagement

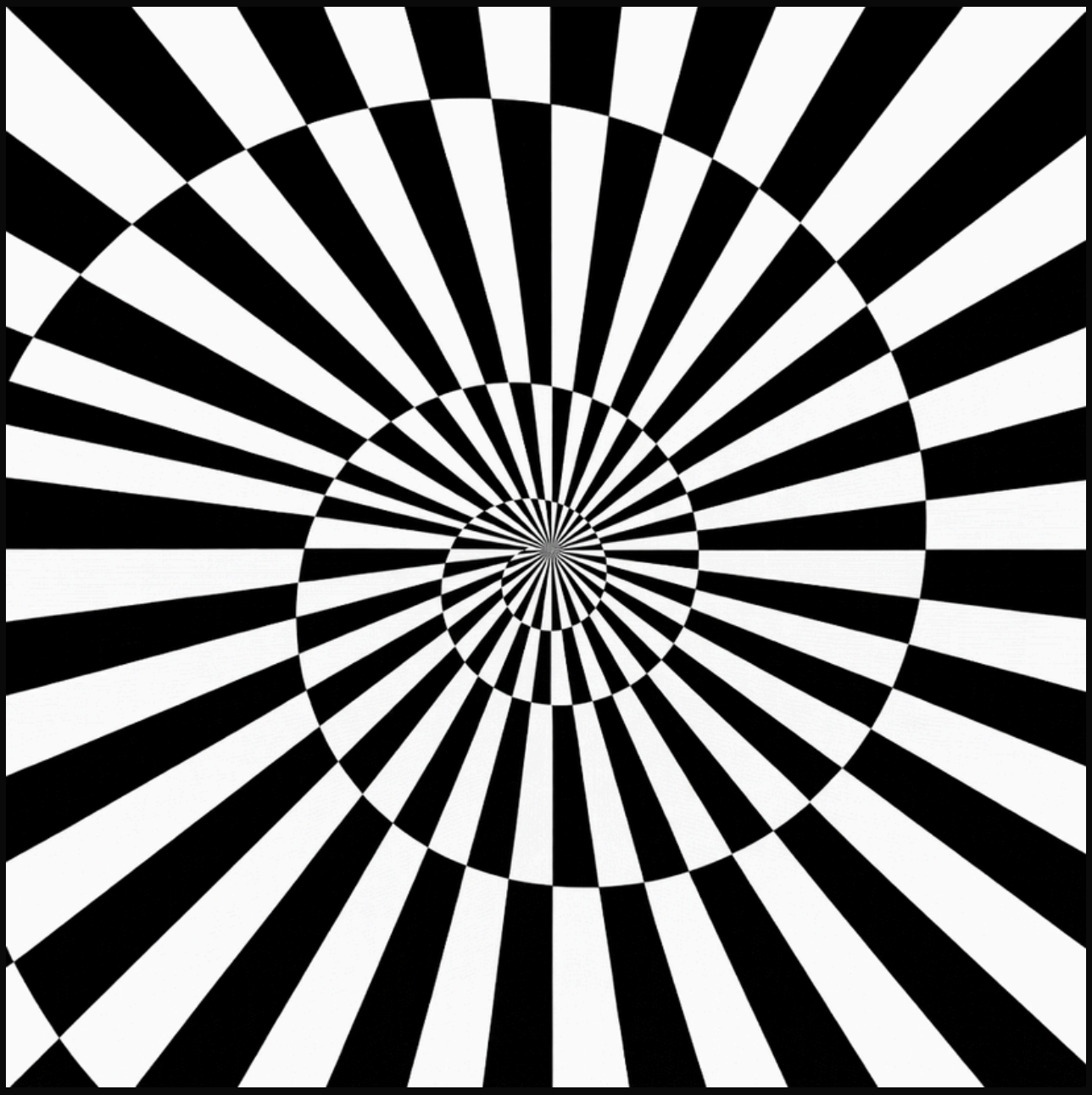


Table of Contents

1. About Inversive Labs
2. Solaris: Testing Methodology
3. Disclaimer
4. Risk Classification
5. Protocol Summary
6. Engagement Timeline
7. Executive Summary
8. Summary of Findings
9. Invariants Tested
10. Detailed Findings

- ILRV-2026-02.01: INV-F1 - Fee Basis Points Can Exceed Maximum
- ILRV-2026-02.02: INV-N1 - Token Freeze Authority Enables DoS

1. About Inversive Labs

Inversive Labs is a boutique security research firm dedicated to securing emerging blockchain ecosystems. We specialize in advanced testing methodologies including stateful fuzzing, invariant testing, and formal verification for complex DeFi protocols.

Prior to Inversive Labs, our founders established themselves as independent security researchers, discovering critical vulnerabilities in production protocols and contributing to the security of major DeFi and open-source projects. Our engineers bring diverse backgrounds in offensive security, low-level exploitation, and smart contract security.

Our expertise spans traditional security and blockchain attack vectors, enabling us to identify critical vulnerabilities across complex protocol implementations and contribute meaningful security research to rapidly evolving ecosystems.

For more on Inversive Labs' security research and services, visit https://inversive.xyz, reach us on X at @inversive_xyz or send us an email at contact@inversive.xyz

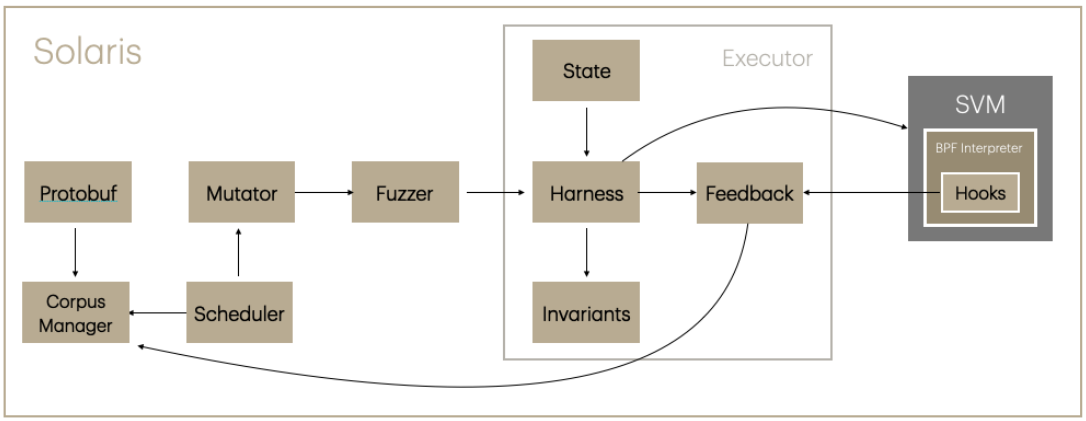
2. Solaris: Testing Methodology

Stateful Structure-Aware Fuzzing for Solana Programs

Solaris is our proprietary stateful, structure-aware fuzzer specifically engineered for Solana programs. By tracking execution state across transaction sequences, understanding program structure, and using bytecode-level coverage feedback, Solaris systematically explores target program state spaces to uncover vulnerabilities that would remain hidden from traditional testing approaches.

What Sets Solaris Apart

Unlike conventional fuzzers that rely on triggering crashes or panics, Solaris focuses exclusively on **business logic violations** and **economic invariants**. This approach is critical for DeFi protocols where subtle state inconsistencies can lead to catastrophic fund loss without any program errors or crashes.



Architecture

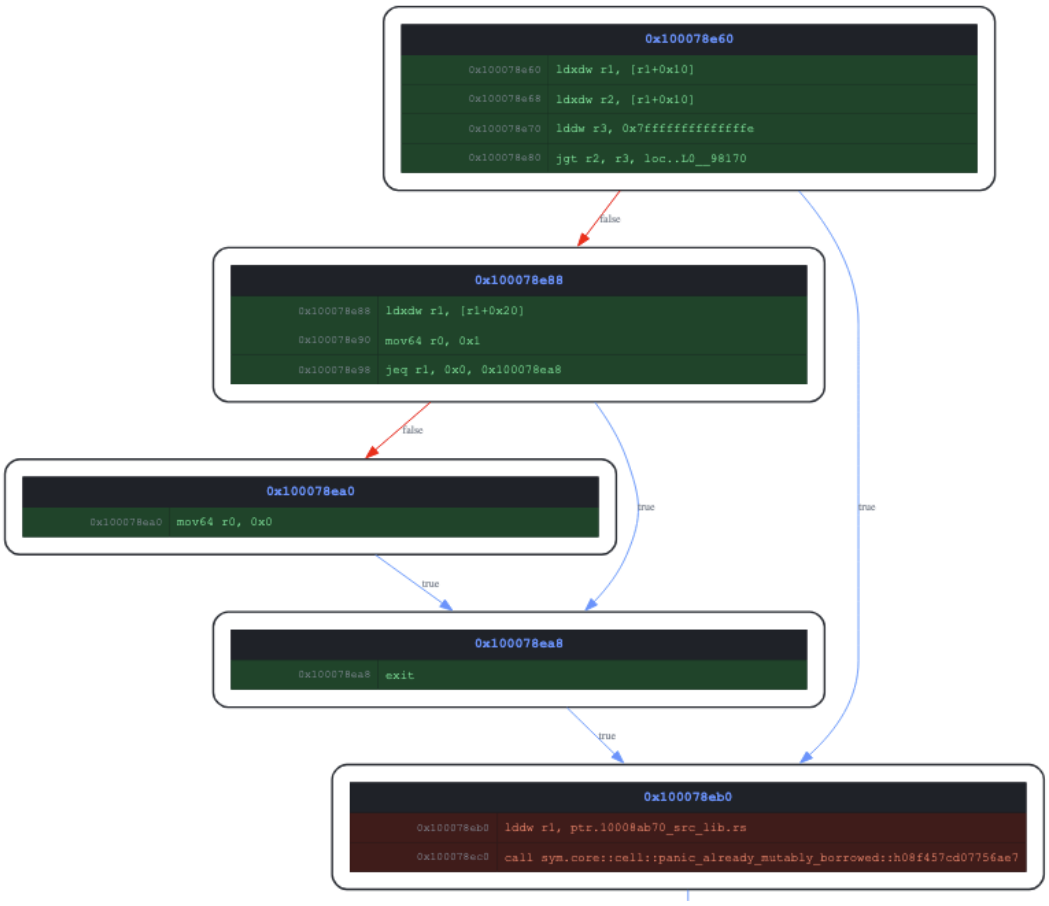
Solaris introduces three groundbreaking innovations in smart contract fuzzing:

1. Structure-Aware Test Generation via Protocol Buffers

We leverage Protocol Buffers to create semantically valid transaction sequences that respect Solana's complex account model and instruction requirements. This structured approach enables:

- **Intelligent Mutation**: Modifications that preserve transaction validity while exploring edge cases
- **Dependency Tracking**: Understanding relationships between accounts and instructions
- **Stateful Sequencing**: Generating transaction chains that build upon previous states

2. Program-Scoped sBPF Coverage Tracking

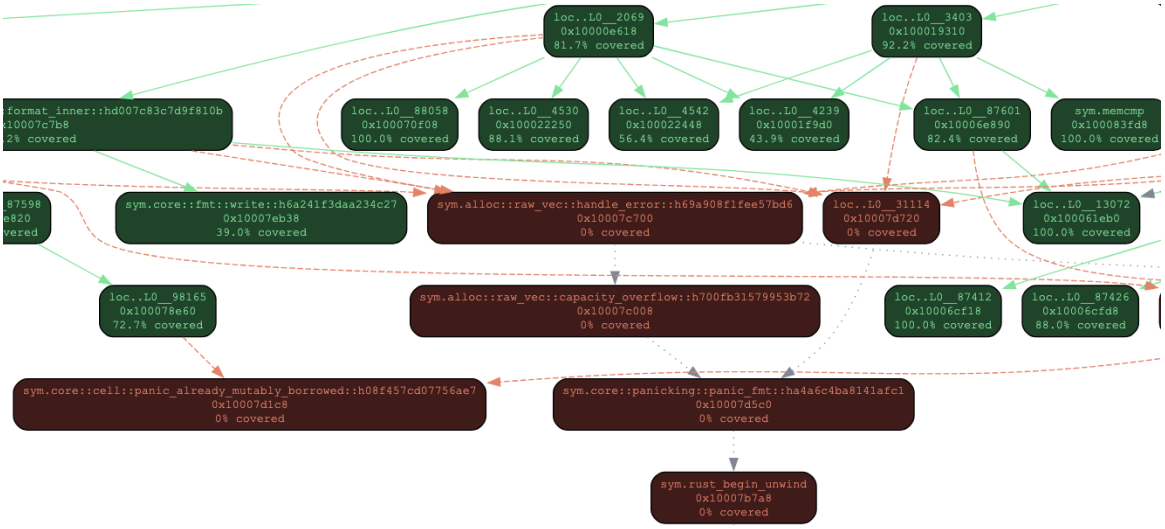


Solaris implements **basic block level granularity** for coverage tracking directly at the sBPF bytecode level ensuring:

- **Precise Coverage Metrics**: Tracking execution at the bytecode level rather than source code
- **CPI Awareness**: Following execution across Cross-Program Invocations
- **Feedback-Driven Evolution**: Using coverage data to guide mutation strategies toward unexplored paths

The control flow graph visualization above demonstrates how Solaris maps and tracks every possible execution path through the program, ensuring enhaustive exploration of code branches.

3. Business Logic Invariant Monitoring



Rather than searching for crashes, Solaris continuously monitors high-level invariants throughout execution. The call graph analysis above illustrates how we track program flow to:

- **Avoid Panic Triggers**: Focus on valid execution paths that maintain program stability
- **Detect Subtle Violations**: Identify state inconsistencies that don't cause crashes
- **Economic Modeling**: Track value flows and conservation laws across operations

Technical Implementation

Solaris implements stateful fuzzing through persistent protocol state management and continuous invariant monitoring. The fuzzer maintains protocol context across test iterations, enabling detection of vulnerabilities that require specific transaction sequences.

- **Coverage Feedback** The fuzzer uses sBPF bytecode coverage feedback to guide test generation toward unexplored execution paths. This approach ensures systematic exploration of the protocol's state space while prioritizing areas with complex branching logic.
- **Invariant Testing** Instead of relying on crash detection, Solaris evaluates user-defined invariants after each transaction. These invariants encode critical protocol properties.
- **Transaction Generation Strategy** Solaris generates valid transaction sequences using Protocol Buffer definitions as a schema to degfine decission trees to ensure program transaction are executed in defined sequences.

This structured approach produces test cases that respect protocol semantics while exploring edge conditions.

3. Disclaimer

This document presents the findings from an invariant testing campaign conducted on Fuul Protocol over a 1-week period. The testing focused on validating business logic invariants and economic properties through systematic fuzzing with Solaris, our stateful sBPF coverage-guided fuzzer.

This report documents vulnerabilities and invariant violations identified during the testing period. The analysis represents the state of the codebase at a specific point in time and does not account for subsequent modifications. Security testing is inherently limited by time, resources, and the evolving nature of attack vectors.

Our testing methodology adheres to industry best practices for security assessment and invariant validation. Findings are classified according to severity based on potential impact and exploitability. All identified issues have been verified through reproduction and root cause analysis.

This report is provided for informational purposes only and does not constitute investment advice, legal counsel, or a guarantee of security. Inversive Labs makes no warranties, express or implied, regarding the absence of vulnerabilities beyond those identified during the testing period. The deployment and use of the analyzed code remain at the sole risk and responsibility of the implementing parties.

4. Risk Classification

Our audit uses the following severity classification system:

SEVERITY	Description
CRITICAL	Issues that can lead to loss of funds, protocol insolvency, or complete system compromise. Immediate action required.
HIGH	Issues that can lead to significant loss of funds or major protocol disruption under certain conditions. Should be fixed promptly.
MEDIUM	Issues that can lead to unexpected behavior, minor loss of funds, or degraded user experience. Should be addressed.
LOW	Issues that represent potential risks or deviations from best practices. Recommended to fix.
INFORMATIONAL	Code quality improvements, gas optimizations, or suggestions for better practices. Optional fixes.

5. Protocol Summary

Fuul is a Solana-based protocol built with the Anchor framework that manages project budgets, token distributions, and multi-signature claim operations. The protocol enables:

- **Project Management:** Creation and administration of projects with role-based access control
- **Budget Management:** Deposit and withdrawal of fungible and non-fungible tokens
- **Claim System:** Multi-signature verification for token claims with cooldown limits
- **Currency Management:** Global token configuration with claim limits and cooldown periods
- **Fee System:** Configurable removal fees with global defaults and project-specific overrides

Key Components:

- Global configuration with role-based permissions
- Project-level budget tracking and token management
- Ed25519 signature verification for multi-party approvals
- Cooldown mechanism to prevent claim abuse
- Associated Token Account (ATA) integration

6. Engagement Timeline

Engagement Timeline

December 18, 2025
Initial meeting and scope definition

Kickoff

Preliminary Audit
ILAR-2026-01

December 22, 2025
Manual Security Audit

December 29, 2025
Initial findings from preliminary audit

First Draft Delivery

Audit Fixes

Dec 30 - Jan 7, 2026
Protocol team implements remediation

Jan 8-15, 2026
Solaris fuzzing campaign begins

Invariant Testing

Testing Fixes

Jan 16-20, 2026
Address invariant testing findings

January 21, 2026
Testing suite and report delivery

Final Report
ILAR-2026-02

7. Executive Summary

Programs Tested

PROGRAM	COMMIT HASH	LINES OF CODE
fuul-protocol/protocol-program-svm	`05bf08cab78d7f3c92d706415c4aea7002d805f7`	~3,200

Invariant Categories

The invariant testing engagement validated the following property categories:

CATEGORY	Definition	COUNT
Balance	**Token conservation, budget correctness, and fund flow validation**	6
Counter	**Monotonicity of nonces, attribution counts, and claim counters**	4
Rate Limiting	**Cooldown enforcement and cumulative claim limits**	3
Fee	**Fee calculation correctness and basis point bounds**	4
Authorization	**Access control, pause enforcement, and multi-sig requirements**	5
Timestamp	**Deadline validation, cooldown expiry, and time monotonicity**	4
State Consistency	**Cross-account relationships and counter synchronization**	5
Mathematical	**Overflow protection, global conservation, and NFT constraints**	5
Special Cases	**Edge cases including first deposit, last admin, and whitelist bypass**	6
Cross-Operation	**Multi-step properties across deposit-claim-remove cycles**	3
Total		**45**

Engagement Overview

Following the completion of manual security audit ILAR-2026-01, Inversive Labs conducted a **1-week** invariant testing campaign on Fuul protocol's Solana program to validate the applied fixes and ensure protocol correctness through systematic fuzzing. The engagement proceeded through two phases:

Phase 1: Fuzzing Suite Development

Built an exhaustive stateful fuzzing suite using **Solaris**, our proprietary sBPF coverage-guided fuzzer for Solana programs, tailored specifically for Fuul's Solana program architecture.

Phase 2: Invariant Definition & Testing

Identified and tested 45 invariants, focusing exclusively on protocol and business logic.

Summary

Throughout the testing campaign, **45 invariants** were continuously validated across millions of stateful test iterations. The fuzzing campaign identified **2 invariant violations** which were promptly reported to the Fuul team for remediation. The complete Solaris test suite has been made publicly available on GitHub for transparency.

8. Summary of Findings

Vulnerability Distribution

SEVERITY	COUNT	FIXED	ACKNOWLEDGED	DISPUTED
CRITICAL	**0**	0	**0**	0
HIGH	**0**	0	**0**	0
MEDIUM	**1**	1	**0**	0
LOW	**1**	0	**1**	0
INFO	**0**	0	**0**	0

Testing Statistics

METRIC	VALUE
Total Test Cases	8,000,000
Invariants Checked	45
Violations Found	2
Test Duration	4 days

9. Invariants Tested

Balance Invariants

ID	Invariant	STATUS	Description
INV-B1	Budget Non-Negativity	PASSED	Project budget must always be greater than or equal to zero
INV-B2	Deposit Increases Budget	PASSED	Deposit operation increases budget by exactly the deposited amount
INV-B3	Claim Decreases Budget Correctly	PASSED	Claim operation decreases budget by claim amount plus project fee
INV-B4	Remove Decreases Budget	PASSED	Remove operation decreases budget by the full removal amount
INV-B5	Conservation of Funds (Claim)	PASSED	Budget decrease equals recipient amount plus fee collector amount
INV-B6	Conservation of Funds (Remove)	PASSED	Budget decrease equals authority receives plus fee collector receives

Counter Invariants

ID	Invariant	STATUS	Description
INV-C1	Project Nonce Monotonicity	PASSED	Global project nonce only increases by one per new project
INV-C2	Attributions Count Monotonicity	PASSED	Project attributions count only increases by one per claim
INV-C3	Total Claims Monotonicity	PASSED	Project user total claims counter only increases
INV-C4	Total Native Claimed Monotonicity	PASSED	Total native claimed only increases for native token claims

Rate Limiting Invariants

ID	Invariant	STATUS	Description
INV-R1	Cumulative Claim Within Limit	PASSED	Cumulative claims per cooldown period never exceed the limit
INV-R2	Claim Amount Within Limit	PASSED	Single claim amount cannot exceed the cooldown period limit
INV-R3	Cooldown Reset Behavior	PASSED	Cooldown expiry resets cumulative counter and updates period start

Fee Invariants

ID	Invariant	STATUS	Description
INV-F1	Fee Basis Points Range	FAILED	All fee configurations must be between 0 and 10000 basis points
INV-F2	Fee Calculation Correctness	PASSED	Fee equals amount times fee basis points divided by 10000
INV-F3	No Zero Fees for Non-Zero Amounts	PASSED	*Behavior matches Ethereum implementation - zero fees allowed by design on small amounts
INV-F4	Project Fee Comes from Budget	PASSED	Project claim fee deducts from budget, not from recipient

Authorization Invariants

ID	Invariant	STATUS	Description
INV-A1	Replay Prevention via Proof	PASSED	A proof cannot be used twice for the same project
INV-A2	Multi-Signature Requirement	PASSED	Valid signers count must meet or exceed required signers
INV-A3	Paused State Blocks Claims	PASSED	All claims must fail when global config is paused
INV-A4	Currency Active Required	PASSED	Currency token must be active for deposits and claims
INV-A5	Only Admins Manage Budgets	PASSED	Deposit and remove operations require admin role

Timestamp Invariants

ID	Invariant	STATUS	Description
INV-T1	Message Deadline Not Expired	PASSED	Claim message deadline must be greater than or equal to current time
INV-T2	Cooldown Period Calculation	PASSED	Cooldown expires when current time exceeds period start plus duration
INV-T3	Timestamps Are Monotonic	PASSED	Claim cooldown period start timestamp can only increase
INV-T4	Attribution Timestamp Accuracy	PASSED	Attribution timestamp must be within tolerance of actual time

State Consistency Invariants

ID	Invariant	STATUS	Description
INV-S1	Budget Exists If Deposits Made	PASSED	Successful deposit must create project currency budget account
INV-S2	Attribution Count Matches	PASSED	Project attributions count equals actual attribution records
INV-S3	ProjectUser Claims Consistency	PASSED	Project user total claims equals actual claims by that user
INV-S4	Active Currency Can Have Budget	PASSED	Budget greater than zero implies currency was active at deposit
INV-S5	Token Account Consistency	PASSED	SPL tokens with balance must have token account configured

Mathematical Invariants

ID	Invariant	STATUS	Description
INV-M1	Sum of All Budgets Conservation	PASSED	Total deposited minus claimed minus removed minus fees equals sum of budgets
INV-M2	Integer Overflow Protection	PASSED	All arithmetic operations use checked methods to prevent overflow
INV-M3	Fee Percentage Bounds	PASSED	Fee amount must be between zero and one hundred percent of amount
INV-M4	NFT Amount Invariants	PASSED	NFT operations require amount of one and zero project claim fee
INV-M5	Transitive Budget Relationship	PASSED	Order relations between budgets preserved after identical operations

Special Case Invariants

ID	Invariant	STATUS	Description
INV-X1	First Deposit Creates Budget	PASSED	First deposit for project and currency creates budget account
INV-X2	Last Admin Cannot Renounce	PASSED	Last admin renounce operation must fail to prevent lockout
INV-X3	Zero Amount Operations Rejected	PASSED	Operations with zero amount must be rejected
INV-X4	Whitelist Bypass for Native Fee	PASSED	Whitelisted users do not pay user native claim fee
INV-X5	Message Domain Validation	PASSED	Claim message domain must match program ID and version
INV-N1	Freeze Authority DoS Protection	FAILED	Currency tokens must not have freeze authority set

Cross-Operation Invariants

ID	Invariant	STATUS	Description
INV-CO1	Deposit-Claim-Remove Cycle	PASSED	Claims plus removals cannot exceed total deposited amount
INV-CO2	Multiple Claims Sum	PASSED	Sum of all claims cannot exceed deposits minus removals
INV-CO3	Fee Accumulation	PASSED	Fee collector balance increases with each fee-bearing operation

10. Detailed Findings

Medium

[ILRV-2026-02.01] INV-F1 - Fee Basis Points Exceeding Maximum

Description

The Fuul protocol allows setting **project-specific fees** that exceed 10000 basis points (100%), which can result in underflow errors when removing funds. While global config fees are properly validated, project fee updates lack this critical validation. This issue was addressed in commit `e6ab670fdc84129c289cca023dcf127120ebc95c`.

Location: `/programs/fuul-solana/src/instructions/project.rs:76-129`

```
// Current implementation - NO validation against BASIS_POINTS maximum
impl<'info> UpdateProjectFees<'info> {
    pub fn update_project_fees(
        &mut self,
        project_nonce: u64,
        user_native_claim_fee: Option<u64>,
        project_claim_fee: Option<u16>,  // Can be > 10000!
        remove_fee: Option<u16>,          // Can be > 10000!
    ) -> Result<()> {
        // ... validation for changes ...

        // Updates project fees WITHOUT checking if they exceed BASIS_POINTS
        if let Some(project_claim_fee) = project_claim_fee {
            if Some(project_claim_fee) != self.project.fee_management.project_claim_fee {
                self.project.fee_management.project_claim_fee = Some(project_claim_fee);
                has_changes = true;
            }
        }

        if let Some(remove_fee) = remove_fee {
            if Some(remove_fee) != self.project.fee_management.remove_fee {
                self.project.fee_management.remove_fee = Some(remove_fee);
                has_changes = true;
            }
        }
        // ...
    }
}
```

Note: The global config fees in `admin.rs` ARE properly validated, but project-specific fee overrides are not. When fees > 100%, the fee calculation fails:

```
// In remove_fungible_token
let remove_fee: u64 = project.get_remove_fee(global_config).into();
let fee = remove_fee.checked_mul(amount)
    .ok_or(FuulError::Overflow)?
    .checked_div(BASIS_POINTS.into())
    .ok_or(FuulError::Overflow)?;

let amount_after_fee = amount.checked_sub(fee).ok_or(FuulError::Underflow)?;
// ^^^ UNDERFLOWS when fee > amount (which happens when fee% > 100%)
```

Impact

- **Protocol DoS**: Setting project fees > 100% blocks remove operations for that specific project with underflow errors
- **Economic griefing**: Global admins with update_project_fees permission can trap project funds by setting excessive fees
- **Project funds trapped**: Project-specific fees > 100% prevent admins from removing unclaimed funds from that project
- **Fee calculation underflow**: When `fee > amount`, the subtraction at line 584 in `project_budget.rs` causes an underflow

Recommendation

```
// File: programs/fuul-solana/src/instructions/project.rs

impl<'info> UpdateProjectFees<'info> {
    pub fn update_project_fees(
        &mut self,
        project_nonce: u64,
        user_native_claim_fee: Option<u64>,
        project_claim_fee: Option<u16>,
        remove_fee: Option<u16>,
    ) -> Result<()> {
      ...

        // Update the project claim fee if provided
        if let Some(project_claim_fee) = project_claim_fee {
+           // Validate the project claim fee doesn't exceed 100%
+           require!(
+               project_claim_fee <= BASIS_POINTS,
+               FuulError::InvalidPercentage
+           );
            if Some(project_claim_fee) != self.project.fee_management.project_claim_fee {
                self.project.fee_management.project_claim_fee = Some(project_claim_fee);
                has_changes = true;
            }
        }

        // Update the remove fee if provided
        if let Some(remove_fee) = remove_fee {
+           // Validate the remove fee doesn't exceed 100%
+           require!(
+               remove_fee <= BASIS_POINTS,
+               FuulError::InvalidPercentage
+           );
            if Some(remove_fee) != self.project.fee_management.remove_fee {
                self.project.fee_management.remove_fee = Some(remove_fee);
                has_changes = true;
            }
        }
        // ...
    }
}
```

Note: The global config fees in `admin.rs` already have proper validation at lines 188-190 and 202-204, so no changes are needed there.

Low

[ILRV-2026-02.02] INV-N1 - SPL Token Freeze Authority Not Validated

Description

The Fuul protocol accepts any SPL token as a currency without validating the freeze authority, allowing tokens whose accounts can be frozen at any time, permanently locking protocol funds.

Location: `/programs/fuul-solana/src/instructions/admin_currencies.rs:91-125`

```rust
// Current implementation - NO freeze authority check
impl<'info> AddCurrencyToken<'info> {
    pub fn add_currency_token(&mut self, token_type: TokenType, claim_limit_per_cooldown: u64) -> Result<()> {
        let is_native = self.token_mint.key() == Pubkey::default();
        let currency_token = &mut self.currency_token;

        // ... validation ...

        if token_type != TokenType::Native {
            // Only unpacks to verify it's a valid Mint, but does NOT check freeze_authority!
            spl_token::state::Mint::unpack(&self.token_mint.try_borrow_data()?)
                .map_err(|_| FuulError::InvalidTokenMint)?;
            // MISSING: Check that mint.freeze_authority.is_none()
        }

        // Adds currency without verifying it cannot be frozen
        currency_token.is_initialized = true;
        currency_token.token_mint = self.token_mint.key();
        currency_token.token_type = token_type.clone();
        // ...
    }
}
```

Impact

- **Funds permanently locked**: Freeze authority can lock all protocol funds of that token type
- **Selective censorship**: Specific projects can be targeted by freezing their token accounts
- **Extortion risk**: Attackers could demand ransom to unfreeze accounts
- **Protocol reputation damage**: Users losing funds due to frozen accounts would damage protocol trustworthiness

Recommendation

```
// File: programs/fuul-solana/src/instructions/admin_currencies.rs

impl<'info> AddCurrencyToken<'info> {
    pub fn add_currency_token(&mut self, token_type: TokenType, claim_limit_per_cooldown: u64) -> Result<()> {
        let is_native = self.token_mint.key() == Pubkey::default();
        let currency_token = &mut self.currency_token;

        // Verify the currency token is not already initialized
        require!(
            !currency_token.is_initialized,
            FuulError::AlreadyInitialized
        );

        if token_type == TokenType::Native && !is_native || token_type != TokenType::Native && is_native {
            return Err(FuulError::InvalidTokenType.into());
        } else if token_type != TokenType::Native {
-           // Try to verify it's a valid Mint
-           spl_token::state::Mint::unpack(&self.token_mint.try_borrow_data()?)
-               .map_err(|_| FuulError::InvalidTokenMint)?;
+           // Unpack and validate the mint account
+           let mint = spl_token::state::Mint::unpack(&self.token_mint.try_borrow_data()?)
+               .map_err(|_| FuulError::InvalidTokenMint)?;
+
+           // Critical: Reject tokens with freeze authority to prevent fund locking
+           require!(
+               mint.freeze_authority.is_none(),
+               FuulError::TokenHasFreezeAuthority
+           );
        }

        currency_token.is_initialized = true;
        currency_token.token_mint = self.token_mint.key();
        currency_token.token_type = token_type.clone();
        currency_token.claim_limit_per_cooldown = claim_limit_per_cooldown;
        // ... rest of function
    }
}
```

Also add the new error variant:

```
// File: programs/fuul-solana/src/errors.rs

#[error_code]
pub enum FuulError {
    // ... existing errors (currently goes up to InvalidPercentage = 6038)

+   #[msg("Token has freeze authority and cannot be used as currency")]
+   TokenHasFreezeAuthority, // 6039
}
```

Additional Recommendations

1. **Whitelist Approach**: Consider maintaining a whitelist of approved tokens that have been vetted for safety
2. **UI Warnings**: Add warnings in the UI when interacting with tokens that have freeze authorities
3. **Documentation**: Clearly document this requirement so projects understand why certain tokens cannot be used
4. **Emergency Mechanism**: Consider implementing an emergency withdrawal mechanism that could be triggered by governance in case of frozen accounts
5. **Monitoring**: Implement monitoring to detect if freeze authorities are added to previously safe tokens